

React Hooks, React Router, Axios

RUJUAN XING



React Hooks: useEffect, useContext

Effect Hook

The `useEffect` Hook allows you to perform side effects in your components.

Some examples of side effects are: fetching data, directly updating the DOM, and timers.

`useEffect` accepts two arguments. The second argument is optional.

`useEffect(<function>, <dependency>)`

Some effects might require cleanup so they return a function:

1. No dependency passed:

```
useEffect(() => { //Runs on every render });
```

2. An empty array:

```
useEffect(() => { //Runs only on the first render }, []);
```

3. Props or state values:

```
useEffect(() => { //Runs on the first render //And any time any dependency value changes }, [prop, state]);
```

useEffect Demo 1

– doesn't depend on anything

Example: Use `setTimeout()` to count 1 second after initial render:

```
export default function Timer() {  
  const [count, setCount] = useState(0);  
  
  setTimeout(() => {  
    setCount((count) => count + 1);  
  }, 1000);  
  
  return <h1>I've rendered {count} times!</h1>;  
}
```



```
export default function Timer() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    setTimeout(() => {  
      setCount((count) => count + 1);  
    }, 1000);  
  }, []);  
  
  return <h1>I've rendered {count} times!</h1>;  
}
```

useEffect Demo 2

- depend on a variable

If there are multiple dependencies, they should be included in the useEffect dependency array.

```
export default function Counter() {  
  const [count, setCount] = useState(0);  
  const [calculation, setCalculation] = useState(0);  
  
  useEffect(() => {  
    setCalculation(() => count * 2);  
  }, [count]); // <- add the count variable here  
  
  return (  
    <>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount((c) => c + 1)}>+</button>  
      <p>Calculation: {calculation}</p>  
    </>  
  );  
}
```

useEffect Demo 3

- Effect cleanup

Some effects require cleanup to reduce memory leaks.

Timeouts, subscriptions, event listeners, and other effects that are no longer needed should be disposed.

We do this by including a return function at the end of the `useEffect` Hook.

```
export default function CleanUp(){
  const [count, setCount] = useState(0);

  useEffect(() => {
    const timerId = setInterval(() => {
      console.log('setInterval...');
      setCount(count + 1);
    }, 1000);
    return () => clearInterval(timerId);
  }, [count]);

  return <h1>I've rendered {count} times!</h1>
}
```

The Problem - "Prop Drilling"

State should be held by the highest parent component in the stack that requires access to the state.

To illustrate, we have many nested components. The component at the top and bottom of the stack need access to the state.

To do this without Context, we will need to pass the state as "props" through each nested component. This is called "prop drilling".

```
export default function Component1() {
  const [user, setUser] = useState("Jesse Hall");

  return (
    <>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 user={user} />
    </>
  );
}

function Component2({ user }) {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 user={user} />
    </>
  );
}
```

```
function Component3({ user }) {
  return (
    <>
      <h1>Component 3</h1>
      <Component4 user={user} />
    </>
  );
}

function Component4({ user }) {
  return (
    <>
      <h1>Component 4</h1>
      <Component5 user={user} />
    </>
  );
}

function Component5({ user }) {
  return (
    <>
      <h1>Component 5</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
```

Using Context

Using Context requires 3 simple steps:

1. creating the context
2. providing the context
3. consuming the context

Step 1: Creating the context

`createContext` lets you create a context that components can provide or read.

```
const someContext = createContext(defaultValue)
```

- `defaultValue`: The value that you want the context to have when there is no matching context provider in the tree above the component that reads context. If you don't have any meaningful default value, specify `null`. The default value is meant as a “last resort” fallback. It is static and never changes over time.
- `someContext`: `createContext` returns a context object.

```
import {createContext} from "react";  
  
export const UserContext = createContext("This is default value.");
```

Step 2: providing the context

`Context.Provider` component available on the context instance is used to provide the context to its child components, no matter how deep they are.

To set the value of context use the `value` prop available on the `<Context.Provider value={value} />`.

```
export default function MyComponent() {  
  const [user, setUser] = useState("Edward");  
  
  return (  
    <UserContext.Provider value={user}>  
      <>  
        <h1>{`Hello ${user}!`}</h1>  
        <Component2 user={user}/>  
      </>  
    </UserContext.Provider>  
  );  
}
```

Step 3: Consuming the context

Consuming the context can be performed in 2 ways:

1. Use the `useContext (Context)` React hook:

```
const value = useContext(SomeContext)
```

- `SomeContext`: The context that you've previously created with `createContext`.
- `useContext` returns the context value for the calling component.

```
export default function Component5() {  
  const user = useContext(UserContext);  
  return (  
    <>  
      <h1>Component 5</h1>  
      <h2>{'Hello ${user} again!'}</h2>  
    </>  
  );  
}
```

Step 3: Consuming the context (cont.)

2. Use a render function supplied as a child to `Context.Consumer` special component available on the context instance

```
export default function Component5() {  
  return (  
    <UserContext.Consumer>  
      {user => (  
        <>  
          <h1>Component 5 Consumer</h1>  
          <h2>{`Hello ${user} again!`}</h2>  
        </>  
      )}  
    </UserContext.Consumer>  
  );  
}
```

React Router 6

REACT



What is Routing?

Routing in reactJS is the mechanism by which we navigate through a website or web-application. Routing can be server-side routing or client-side routing. However, React Router is a part of client-side routing.

Server-side routing results in every request being a full page refresh loading unnecessary and repeated data every time causing a bad UX and delay in loading.

With client-side routing, it could be the rendering of a new component where routing is handled internally by JavaScript. The component is rendered only without the server being reloaded.

Why React Router?

React Routing without any knowledge of it can be manually implemented using useState and JSX for conditioning.

```
const [page, setpage] = useState("products")
const routeTo = (newpage) => {
  setpage(newpage)
}
<button onClick={() = routeTo("cart")}> Cart />
{page === "cart" && ( <Cart cart={cart} setcart={setcart}/>)}
```

But large-scale application like e-commerce?

What is React Router?

React Router is a fully-featured routing solution for React apps.

- It offers pre-developed components, Hooks, and utility functions to create modern routing strategies.
- It offers various router mechanisms for the web version, such as URL path-based routing, URL-hash routing, and invisible routing (known as memory routing).
- `npm install react-router-dom`

React Router Packages

React Router is a monorepo project containing five different npm packages, each of which has a different purpose.

- **react-router-dom**: This package offers routing features for React web applications by importing code from the react-router and @remix-run/router packages.
- **react-router-native**: React Native apps can import routing features from this package. react-router-native also uses core logic from react-router and @remix-run/router packages
- **react-router**: This package implements common core logic for react-router-dom and react-router-native packages by including React-specific code
- **@remix-run/router**: This is a frontend-library-agnostic implementation of the application routing concept. Other packages import this package's code to access the low-level implementation of routing. The developer community implemented React Router-like routing for Vue.js and Svelte with @remix-run/router by porting react-router-dom
- **react-router-dom-v5-compat**: This package helps to do an incremental v5-to-v6 migration process by running v5 and v6 parallelly, so developers can migrate component-by-component without modifying the whole codebase at once

Different Routers

The react-router-dom package offers three higher-level, ready-to-use router components:

- **BrowserRouter**: It is the most common type and most used type of router that uses HTML5 history API (pushState, replaceState, and the popstate event) making your UI in sync with the URL.
- **HashRouter**: This type of router uses the hash portion of the URL keeping your UI in sync with the URL (i.e., index.html#/profile)
- **MemoryRouter**: A router that keeps the history of your “URL” in memory does not like to put it in the address bar. Often used but useful in testing and non-browser environments like React Native.

React Router Components

Routes: It's a new component introduced in the v6. It includes features like relative routing and linking, automatic route ranking, nested routes, and layouts.

Route: It is responsible for rendering the UI of a React component. It has a prop called `path` which always matches the current URL of the application. The second required prop is called `element` that tells the `Route` component when a current URL is encountered and which React component to be rendered.

Link: The link component is used to create links to different routes and implement navigation around the application. It works like an HTML anchor tag.

React Router Basic Demo

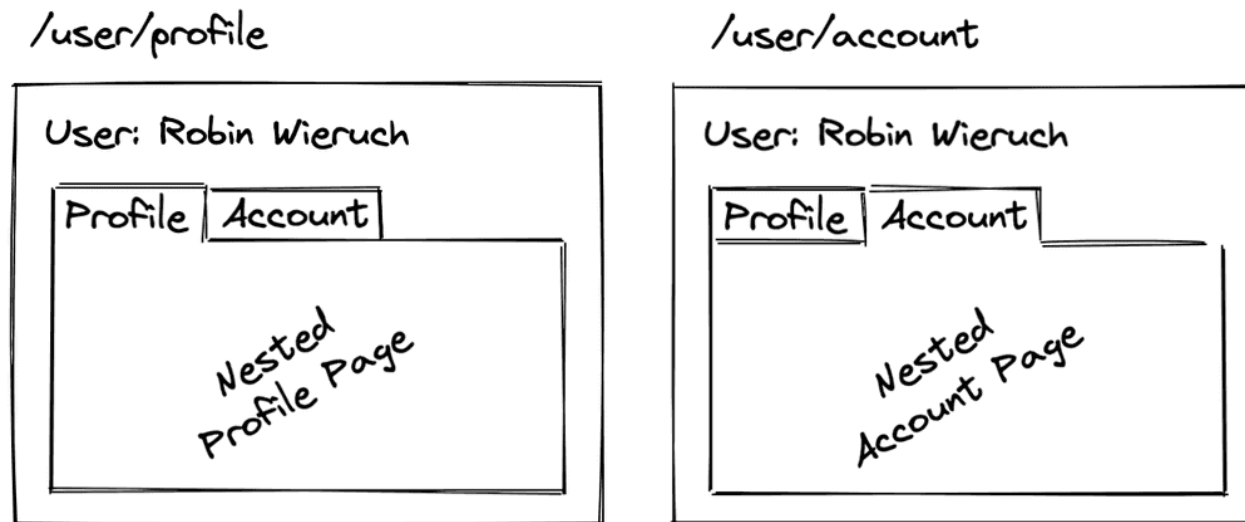
```
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
```

```
function App() {  
  return (  
    <Router>  
      <nav style={{ margin: 10 }}>  
        <Link to="/" style={{ padding: 5 }}>  
          Home  
        </Link>  
        <Link to="/about" style={{ padding: 5 }}>  
          About  
        </Link>  
      </nav>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="*" element={<NoMatch />} />  
      </Routes>  
    </Router>  
  );  
}
```

Nested Routes

Nested Routing is the general idea of coupling segments of the URL to component hierarchy and data.

For example, on a user page one gets presented multiple tabs (e.g. Profile, Account) to navigate through a user's information. By clicking these tabs, the URL in the browser will change, but instead of replacing the whole page, only the content of the tab gets replaced.



useParams

The `useParams` hook returns an object of key/value pairs of the dynamic params from the current URL that were matched by the `<Route path>`.

Returns an object that maps the names of URL Parameters to their values in the current URL.

```
<Route path="/posts" element={<Posts />}>  
  <Route index element={<PostLists />} />  
  <Route path=":id" element={<Post />} />  
</Route>
```

```
const { id } = useParams();
```

Nested Routes Demo

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/posts" element={<Posts />}>
    <Route index element={<PostLists />} />
    <Route path=":id" element={<Post />} />
  </Route>
  <Route path="/about" element={<About />} />
  <Route path="*" element={<NoMatch />} />
</Routes>
```

```
import { Outlet } from 'react-router-dom';
export default function Posts(){

  return (
    <div style={{ padding: 20 }}>
      <h2>Blog</h2>
      <Outlet/>
    </div>
  );
}
```

```
import { Link } from "react-router-dom";

import { blogPosts } from '../data';

export default function PostLists() {

  return (
    <ul>
      {blogPosts.map(b => (
        <li key={b.id}>
          <Link to={`/${posts}/${b.id}`}>
            <h3>{b.title}</h3>
          </Link>
        </li>
      ))}
    </ul>
  );
}
```

```
export default function Post() {
  const { id } = useParams();
  const post = blogPosts.find(b => b.id === id);

  if(!post) {
    return <span>The blog post you've requested
doesn't exist.</span>;
  }
  const { title, description } = post;
  return (
    <div style={{ padding: 20 }}>
      <h3>{title}</h3>
      <p>{description}</p>
    </div>
  );
}
```

useRoutes

The `useRoutes` hook is the functional equivalent of `<Routes>`, but it uses JavaScript objects instead of `<Route>` elements to define your routes. These objects have the same properties as normal `<Route>` elements, but they don't require JSX.

The return value of `useRoutes` is either a valid React element you can use to render the route tree, or null if nothing matched.

useRoutes Demo

```
export default function Routes() {  
  const element = useRoutes([  
    {path: "/", element: <Home/>},  
    {  
      path: "/posts",  
      element: <Posts/>,  
      children: [  
        {index: true, element: <PostLists/>},  
        {path: ":id", element: <Post/>}  
      ],  
    },  
    {path: "/about", element: <About/>},  
    {path: "*", element: <NoMatch/>}  
  ]);  
  return element;  
}
```

```
function App() {  
  return (  
    <Router>  
      <nav style={{ margin: 10 }}>  
        <Link to="/" style={{ padding: 5 }}>  
          Home  
        </Link>  
        <Link to="/posts" style={{ padding: 5 }}>  
          Posts  
        </Link>  
        <Link to="/about" style={{ padding: 5 }}>  
          About  
        </Link>  
      </nav>  
      <MyRoutes />  
    </Router>  
  );  
}
```

Axios

REACT



Axios

Axios is a promise-based HTTP Client for node.js and the browser. It is isomorphic (= it can run in the browser and nodejs with the same codebase). On the server-side it uses the native node.js http module, while on the client (browser) it uses XMLHttpRequests.

It's similar to JavaScript Fetch API.

Features:

- Make XMLHttpRequests from the browser
 - Make http requests from node.js
 - Supports the Promise API
 - Intercept request and response
 - Transform request and response data
 - ...
-
- To use Axios with React, install first: `npm install axios`

Request method aliases

`axios.request(config)`

`axios.get(url[, config])`

`axios.delete(url[, config])`

`axios.post(url[, data[, config]])`

`axios.put(url[, data[, config]])`

Demos

```
try{
  const response = await
  axios.post('http://localhost:8080/students', postData);
  setPostResult(response.data);
  setLoading(false);
} catch(err){
  setError(err);
  setLoading(false);
}
```

```
useEffect(() => {
  axios.get('http://localhost:8080/students')
    .then(response => setStudents(response.data))
}, []);
```

The Axios Instance

- You can create custom Axios instances with specific configurations, base URLs, and interceptors.
- Helpful for managing different parts of your application with distinct settings.

```
axios.create([config])
```

```
const instance = axios.create({  
  baseURL: 'https://some-domain.com/api/',  
  timeout: 1000,  
  headers: {'X-Custom-Header': 'foobar'}  
});
```

Request Config

These are the available config options for making requests. Only the `url` is required. Requests will default to `GET` if method is not specified.

https://axios-http.com/docs/req_config

```
{
  // `url` is the server URL that will be used for the request
  url: 'user',
  method: 'get', // default
  // `baseURL` will be prepended to `url` unless `url` is
  // absolute.
  // It can be convenient to set `baseURL` for an instance of
  // axios to pass relative URLs
  // to methods of that instance.
  baseURL: 'https://some-domain.com/api',
  headers: {
    'X-Requested-With': 'XMLHttpRequest'
  },
  params: { ID: 12345 },

  data: { firstName: 'Fred' },

  data: 'Country=Brasil&City=Belo Horizonte',

  timeout: 1000, // default is `0` (no timeout)

  responseType: 'json' // default
}
```

Response Schema

The response for a request contains the following information.

```
{  
  // `data` is the response that was provided by the server  
  data: {},  
  status: 200,  
  statusText: 'OK',  
  headers: {},  
  
  // `config` is the config that was provided to `axios` for the request  
  config: {},  
  
  // `request` is the request that generated this response  
  // It is the last ClientRequest instance in node.js (in redirects)  
  // and an XMLHttpRequest instance in the browser  
  request: {}  
}
```

```
axios.get('/user/12345')  
  .then(function (response) {  
    console.log(response.data);  
    console.log(response.status);  
    console.log(response.statusText);  
    console.log(response.headers);  
    console.log(response.config);  
  });
```

Refactor Code

http-common.js

```
import axios from "axios";

export default axios.create({
  baseURL: "http://localhost:8080",
  headers: {
    "Content-type": "application/json"
  }
});
```

```
import http from "../http-common";

const getAll = () => {
  return http.get("/students");
};

const get = id => {
  return http.get(`/students/${id}`);
};

const create = data => {
  return http.post("/students", data);
};

const update = (id, data) => {
  return http.put(`/students/${id}`, data);
};

const remove = id => {
  return http.delete(`/students/${id}`);
};

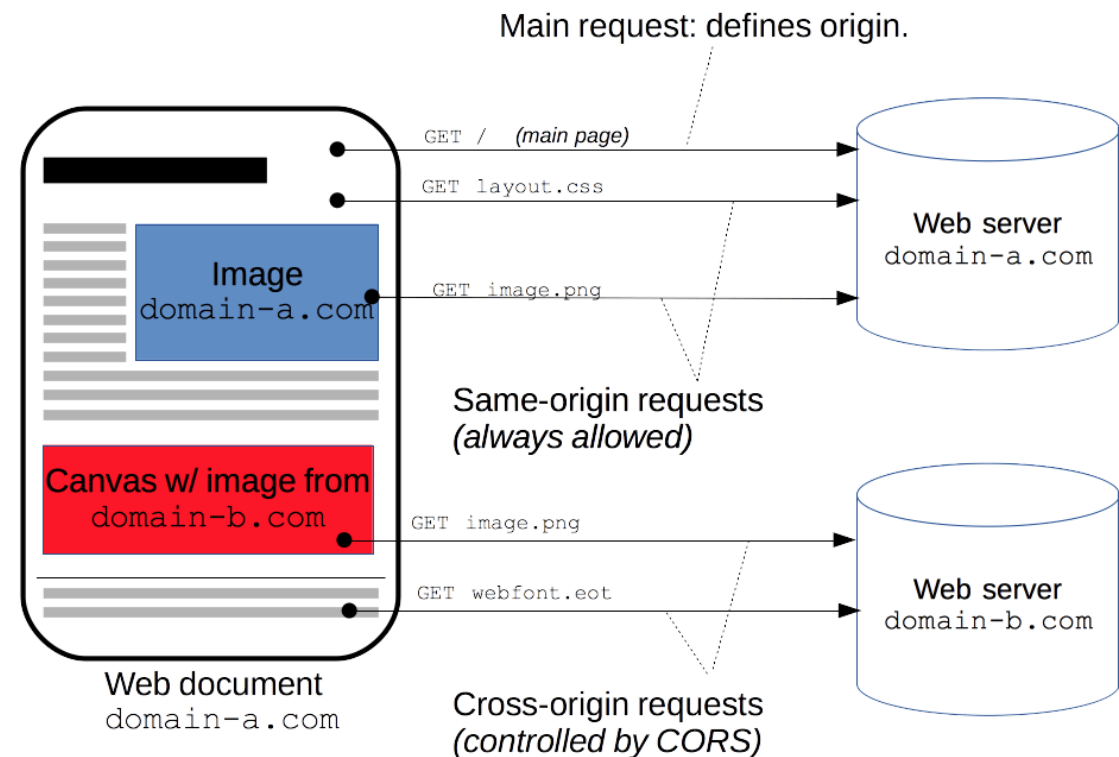
const StudentService = {
  getAll,
  get,
  create,
  update,
  remove
};

export default StudentService;
```

CORS

Cross-Origin Resource Sharing (CORS) is an HTTP-header based mechanism that allows a server to indicate any origins (domain, scheme, or port) other than its own from which a browser should permit loading resources.

CORS also relies on a mechanism by which browsers make a "preflight" request to the server hosting the cross-origin resource, in order to check that the server will permit the actual request. In that preflight, the browser sends headers that indicate the HTTP method and headers that will be used in the actual request.



Server-Side Config

```
@Configuration
public class WebConfig
{
    @Bean
    public WebMvcConfigurer corsConfigurer()
    {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/**")
                    .allowedOrigins("http://localhost:3000")
                    .allowedMethods("*");
            }
        };
    }
}
```

Global CORS Configuration

```
@RequiredArgsConstructor
@RestController
@RequestMapping("/students")
@CrossOrigin(maxAge = 3600)
public class StudentController {

    private final StudentService studentService;

    @GetMapping
    @CrossOrigin("http://example.com")
    public List<Student> getStudents(){
        return studentService.findAll();
    }
}
```

@CrossOrigin on Controller and
Handler Method

Main Points

All websites have something in common: they are made of pages that share similar structures.

A facet of SCI is that Order is found everywhere.

Transcendental consciousness leads to increased adaptability and flexibility.